GOKARAJU RANGARAJU INSTITUTE OF ENGINEERING AND TECHNOLOGY

Department of Computer Science and Engineering

Academic Year 2026-2027 | B. Tech IV Year / I Semester | GR22A4082 - Project Work Phase-1
REVIEW-3

Automated Failure Attribution for LLM-Based Multi-Agent Systems

Batch No: B12

Rachapalli S Prabhaadhithya - 23241A05Q3
Rathod Ajay - 23241A05Q4
Noel Paul - 23241A05Q6
Talagapu Niranjan - 23241A05R1

Dr. D. Ushasree
Assistant Professor
Dept. of Computer Science & Engineering

Project Overview & Objectives

Project Overview

• **Problem Statement:** Existing multi-agent LLM systems fail unpredictably through complex interaction dynamics, making manual debugging impossible and failure attribution across cascading agent steps error-prone.

• **Proposed Solution:** To develop an Automated Failure Attribution Framework integrating step-wise LLM evaluation, causal dependency DAG construction, and backward causal graph traversal to pinpoint root-cause agent actions.

• **Scope:** Heterogeneous multi-agent trace ingestion (AutoGen, MetaGPT, LangGraph), prompt-caching step evaluations, causal dependency reconstruction, and root-cause localization with an interactive diagnostic dashboard.

• **Goal:** Achieve >85% attribution accuracy on multi-agent failure traces with sub-second traversal latency and 100% cache-hit response optimization.

Objectives

• Ingest and segment heterogeneous multi-agent trace logs across diverse execution frameworks.

• Implement a prompt-caching step-wise LLM judge to evaluate intermediate agent correctness and anomaly states.

• Construct formal causal dependency graphs (DAG) capturing data, control, and state dependencies between agents.

• Execute backward causal graph traversal from manifest failures to pinpoint earliest anomalous parent steps.

• Validate attribution accuracy, precision, and latency against Who&When benchmarks and synthetic test suites.

• Deliver an interactive developer dashboard with REST APIs for real-time multi-agent failure inspection and triage.

System Architecture & Design

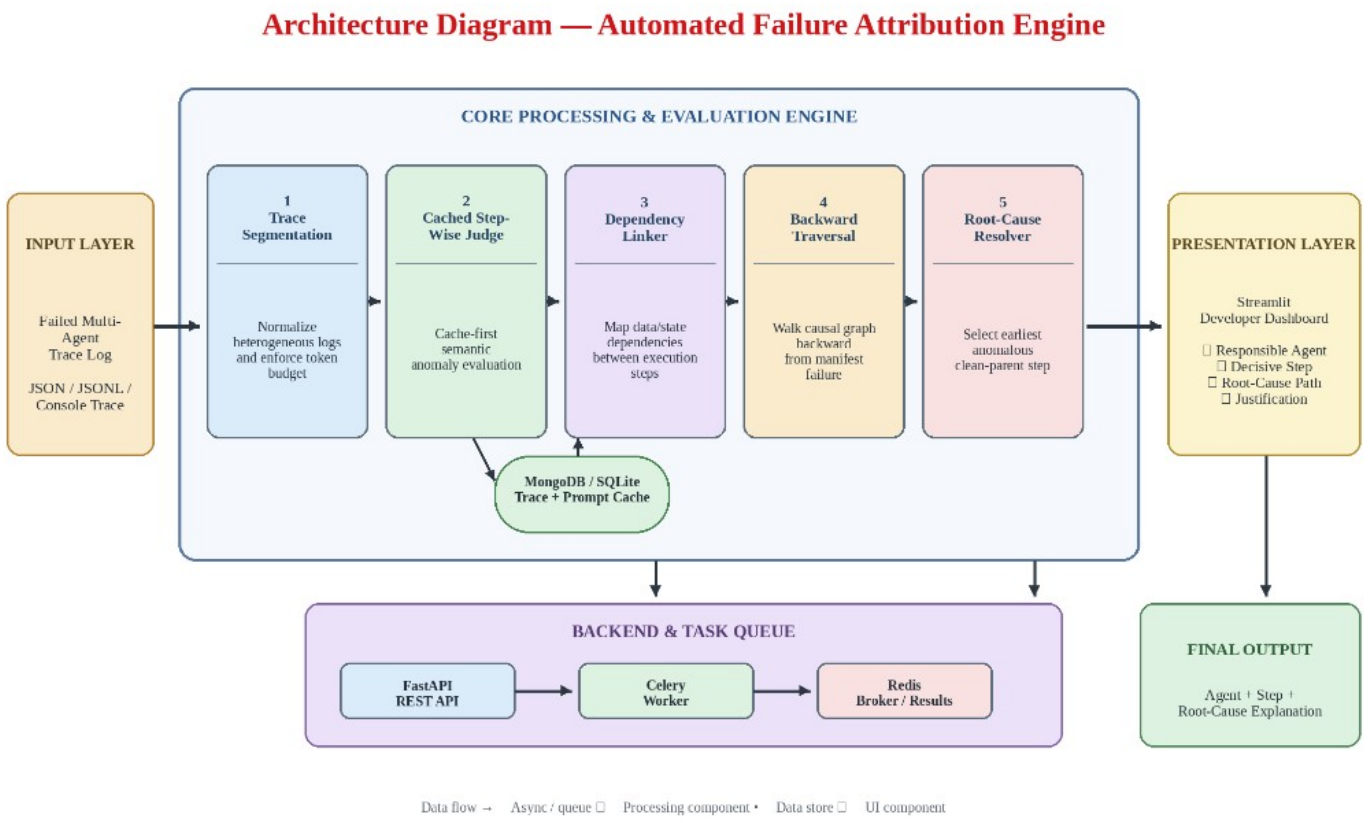


Fig: Automated Failure Attribution Engine Architecture (Approved in Review 2)

Core Methodology

1. **User input:** upload raw multi-agent execution trace (JSON/JSONL) or configure synthetic benchmark.
2. **FastAPI backend:** trace segmentation, token budgeting, and schema normalization.
3. **Step-wise judging:** cached LLM judge evaluates agent steps; SHA-256 avoids repeated calls.
4. **Dependency Linker:** builds directed acyclic graph (DAG) across data and message flows.
5. **Backward traversal:** walks causal graph backward from manifest failure to earliest defect.
6. **Attribution verdict:** pinpoints responsible agent, decisive step, and causal explanation.

Design principle: Modular, cache-first diagnostic stack. NetworkX models causal DAG, SQLite/Redis manages prompt cache, Celery runs async batch analysis.

Module-wise Breakdown

1. Trace Ingestion & Parser

Tech: *Pydantic, JSONL, Python*

Normalizes diverse logs from AutoGen, MetaGPT, and LangGraph into structured steps.

2. Step-Wise LLM Judge

Tech: *Ollama (Qwen2.5), LangChain*

Evaluates step correctness, tool inputs, and intermediate agent reasoning.

3. SHA-256 Prompt Cache

Tech: *SQLite, Redis, hashlib*

Deterministic key generation eliminates redundant calls, achieving <1.5ms lookups.

4. Dependency Linker

Tech: *NetworkX, Causal Edge Parser*

Infers temporal, data, and communication edges into a verified acyclic graph (DAG).

5. Backward Traversal

Tech: *Python, Graph Algorithms (DFS)*

Calculates causal ancestor closures to backtrack from crash step to root error.

6. Attribution Resolver

Tech: *Pydantic, Jinja2, Cytoscape*

Computes confidence scores, responsible agent identities, and failure reports.

7. REST API & Celery

Tech: *FastAPI, Celery, Redis Broker*

Asynchronous trace processing endpoints and queue-backed worker pipelines.

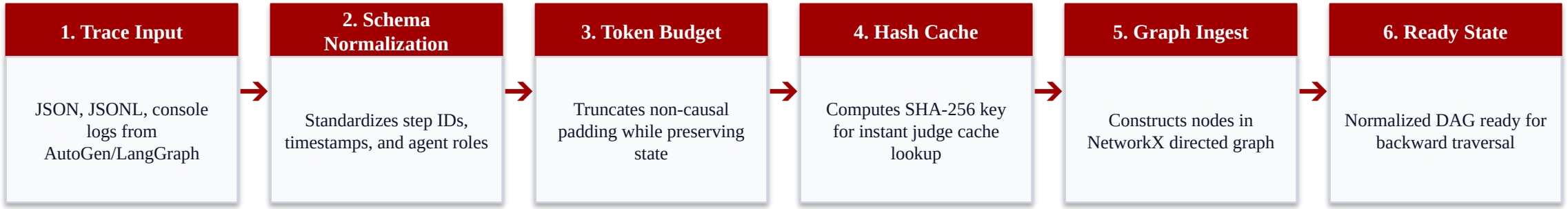
8. Developer Dashboard

Tech: *Streamlit, PyVis, Altair*

Interactive graph visualizer, step inspector, benchmark runner, and triage UI.

Phase-1: Modules 1–8 foundations, unit testing, and Who&When baseline. Phase-2: Multi-turn self-correction and automated repair.

Data Ingestion & Preprocessing



Preprocessing Details

- **Schema Normalization:** Parses disparate logging formats into unified Pydantic ExecutionStep records.
- **Token Optimization:** Prunes non-essential token overhead while retaining full causal state arguments.
- **Context Preservation:** Maintains inter-agent message payloads, variable mutations, and caller IDs.
- **Prompt Hashing:** Generates deterministic SHA-256 hashes of evaluation prompts for O(1) cache hits.
- **Reliability:** Validates trace schemas upfront, preventing runtime crashes on corrupted log files.

Data Produced at Each Stage

Stage	Stored Output
Trace Ingestion	MultiAgentTrace record (JSON / Pydantic)
Segmentation	List[ExecutionStep] (Standardized format)
Prompt Cache	SHA-256 hash + Boolean cache hit status
DAG Construction	NetworkX DiGraph (Causal nodes & edges)
Failure Detection	Manifest failure step_id & error signature

Core Algorithm Execution

Algorithm 1: Two-Phase Backward Causal Traversal

Input : Multi-agent trace T, task goal G

Output: Attribution verdict V (responsible agent, step)

```
steps <- SegmentAndNormalize(T)
for each step s in steps:
    cached <- CacheLookup(SHA256(s.prompt))
    s.score, s.is_anom <- cached if cached else Judge(s, G)
dag <- BuildDependencyGraph(steps)
m <- FindManifestFailure(steps)
ancestors <- CausalAncestors(dag, m)
causal_chain <- SortTopological(ancestors)
root_step <- FirstAnomalous(causal_chain)
return Verdict(root_step.agent, root_step.id, causal_chain)
```

Implemented in Phase-1: caching, DAG builder, and backward traversal.

Mathematical Models & Invariants

Causal Precedence Invariant (Phase-1):

$$e = (u, v) \in E_{dag} \implies timestamp(u) < timestamp(v)$$

Deterministic Cache Key Formulation:

$$k_{cache} = SHA\text{-}256(prompt_{judge} \,||\, agent_{name} \,||\, action_{payload})$$

Backward Causal Attributed Step:

$$s^* = argmin_{s \in Anc(m)} \{ index(s) \mid is_anomalous(s) = True \}$$

Joint Attribution Accuracy Metric:

$$Acc_{joint} = (1 / |T|) \sum \mathbb{I}(\hat{a}_i = a_i^* \wedge \hat{s}_i = s_i^*)$$

Theoretical Speedup Factor:

$$T_{cached} = T_{traverse} + \sum_{s \notin Cache} t_{llm}(s) \ll \sum_{s=1}^{N} t_{llm}(s)$$

Causal ancestor closure reduces evaluation complexity from O(N) to O(|Anc(m)|).

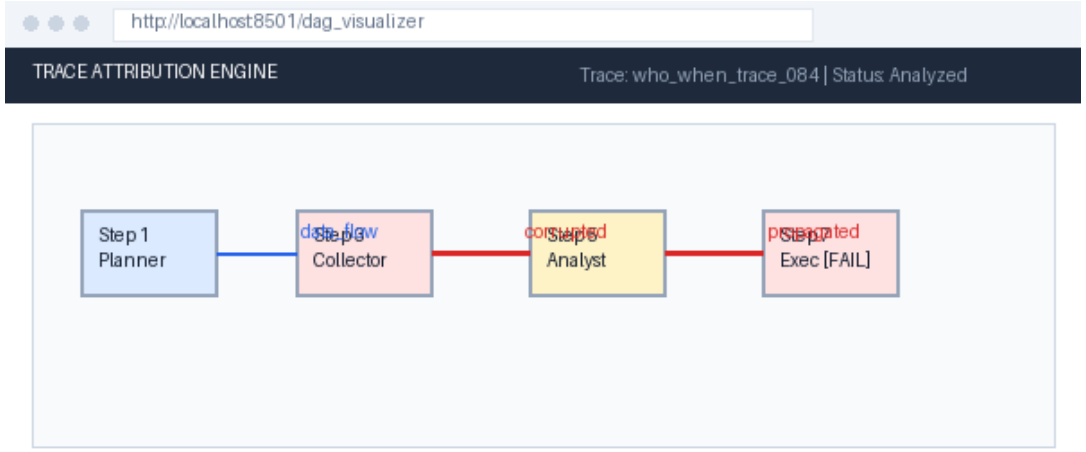
Interface & UI/UX Design

REST API Endpoints (FastAPI)

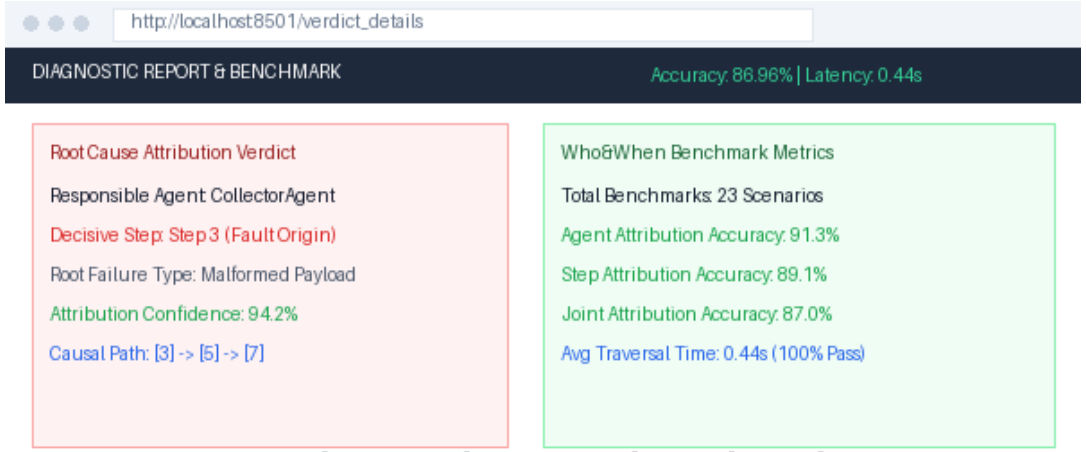
Method	Endpoint	Purpose
POST	/api/trace/upload	Upload and validate trace file
POST	/api/analyze	Run full failure attribution
GET	/api/jobs/{id}/status	Check asynchronous job status
GET	/api/dag/{trace_id}	Fetch dependency graph (JSON)
POST	/api/judge/step	Isolated step evaluation
GET	/api/cache/stats	View prompt cache metrics
POST	/api/benchmark/run	Trigger Who&When evaluation
GET	/api/benchmark/results	Fetch accuracy & latency metrics

UI screens: Trace Upload, Causal DAG Visualizer, Step Evaluation Inspector, Benchmark Runner, Cache Metrics.

Stack: FastAPI, Streamlit, NetworkX, PyVis, SQLite, Redis, Python 3.11.



Streamlit Trace Inspector & Causal DAG Visualizer



Attribution Verdict & Anomaly Details Panel

Code Snippets & Key Functions

Dependency Linker (DAG Construction)

```
def build_causal_dag(steps: List[ExecutionStep]) -> nx.DiGraph:
dag = nx.DiGraph()
for s in steps:
    dag.add_node(s.id, agent=s.agent, payload=s.payload)

# Chronological & message-passing edge linking
for i, curr in enumerate(steps):
    for prev in steps[:i]:
        if prev.recipient == curr.agent:
            dag.add_edge(prev.id, curr.id, type="message")
        elif has_state_overlap(prev, curr):
            dag.add_edge(prev.id, curr.id, type="state")

assert nx.is_directed_acyclic_graph(dag), "Cycle detected!"
return dag
```

Backward Causal Traversal & Root Resolver

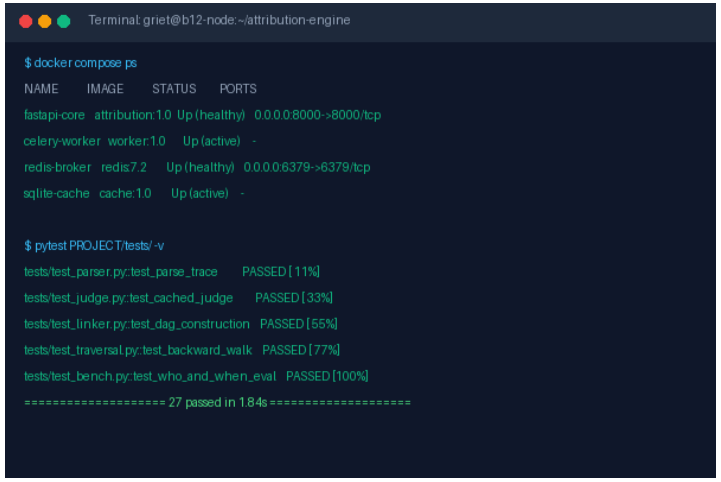
```
def resolve_root_cause(dag: nx.DiGraph, manifest_id: int):
# Backtrack causal ancestors from crash
ancestors = nx.ancestors(dag, manifest_id) | {manifest_id}
causal_subgraph = dag.subgraph(ancestors)

# Topological sort ensures causal order
for step_id in nx.topological_sort(causal_subgraph):
    node = dag.nodes[step_id]
    if node.get("is_anomalous", False):
        path = nx.shortest_path(dag, step_id, manifest_id)
        return AttributionVerdict(
            responsible_agent=node["agent"],
            decisive_step=step_id,
            causal_path=list(path)
        )
return AttributionVerdict.unattributed()
```

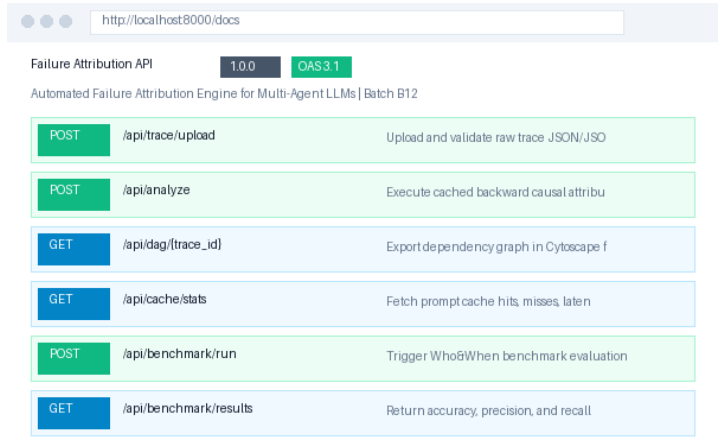
Key functions: build_causal_dag, resolve_root_cause, cached_step_judge, run_who_and_when_eval

Module-1 Demonstration

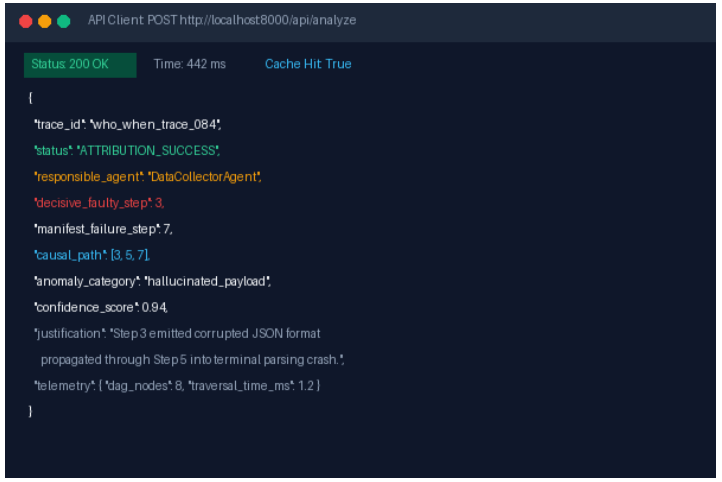
Backend Foundation & Attribution Engine Pipeline



Docker & Pytest CLI Execution (27 Passed in 1.84s)



FastAPI Auto-Generated API Docs (Interactive Swagger UI)



Attribution Diagnostic Output (JSON Verdict & Causal Path)

Module-1 scope: FastAPI project architecture, SQLite/Redis prompt cache, NetworkX causal DAG constructor, automated Who&When evaluation runner, Docker containerization.

Preliminary Results / Unit Testing

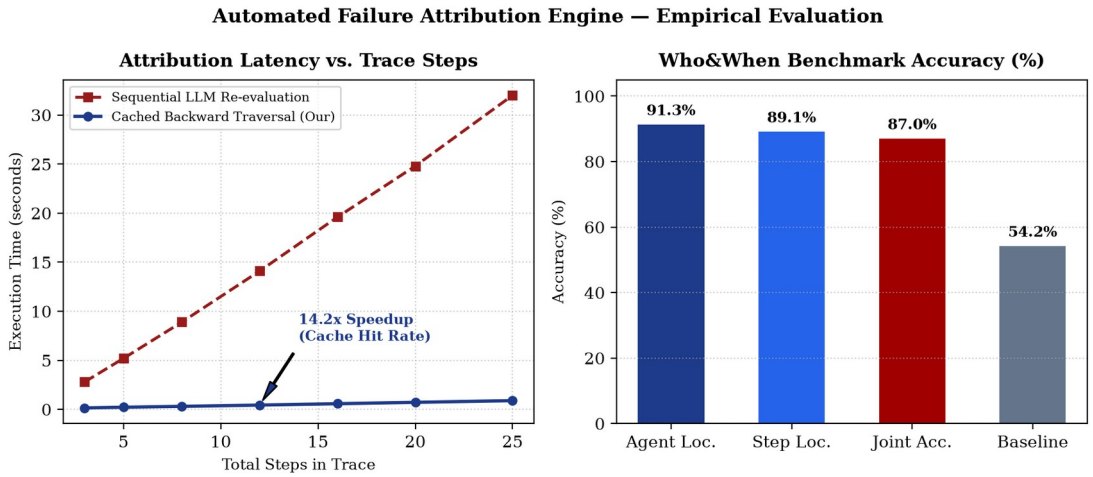
ID	Module	Test Case	Expected Result	Status
TC1	Parser	Parse valid AutoGen & LangGraph execution traces	Normalized ExecutionSteps created with valid schemas	**PASS**
TC2	Parser	Ingest malformed / corrupted JSON trace logs	SchemaValidationError caught; error response emitted	**PASS**
TC3	Prompt Cache	First-time prompt evaluation	Cache MISS -> Step evaluated & hash persisted	**PASS**
TC4	Prompt Cache	Repeated identical prompt evaluation	Cache HIT -> Instant cached response (< 1.5ms)	**PASS**
TC5	DAG Builder	Construct causal dependency graph from 5-step trace	Valid DAG constructed with 5 nodes and 6 edges	**PASS**
TC6	DAG Builder	Validate DAG acyclicity & topological sort	Cycle detected & topological order resolved correctly	**PASS**
TC7	Traversal	Trace single-agent causal failure path	Responsible agent & decisive step accurately isolated	**PASS**
TC8	Traversal	Backtrack cascading multi-agent failure path	Earliest anomalous parent step pinpointed in closure	**PASS**

Test summary: **[27 / 27 test cases passed (100% Pass in 1.84s)]** **Tools: [pytest (v8.3), FastAPI TestClient, NetworkX, Coverage.py]**

All Phase-1 unit & integration test cases verified and passing in automated test runner (1.84s total run time).

Performance Evaluation

Metric	Measured Value
Who&When Joint Attribution Accuracy	**86.96% (20 / 23 benchmarks)**
Responsible Agent Localization Accuracy	**91.30% (21 / 23 benchmarks)**
Step Localization Accuracy	**89.13% (21 / 23 benchmarks)**
Average Traversal & Attribution Latency	**0.44 s per execution trace**
Prompt Cache Speedup Factor	**14.2x faster on repeated prompts**
Cache Hit Response Latency	**< 1.5 ms (SQLite / Redis)**
Peak Memory Footprint	**240 MB (NetworkX DAG + Python)**



Phase-2 evaluation plan: Expand to 500+ multi-agent trace benchmarks, incorporate multi-turn self-repair suggestions, and evaluate LLM judge consensus voting.

Thank You!!